

Trabajo Práctico — Algoritmos

Sistema de Gestión Hospitalaria

Descripción General

El sistema debe permitir gestionar una red de hospitales, pacientes, turnos médicos y derivaciones entre establecimientos de salud de distintas ciudades. Se deberá implementar en C++ de forma completamente orientada a objetos, utilizando estructuras de datos eficientes, manejo adecuado de memoria dinámica, y aplicando los algoritmos estudiados durante la cursada. Se espera un diseño robusto que contemple casos borde y justificación algorítmica documentada.

Archivos de Entrada

hospitales.txt

Cada línea contiene:

```
codigo nombre ciudad capacidad_camas especialidades personal_medico  
presupuesto_anual
```

Ejemplo:

```
HGA HospitalGarrahan BuenosAires 500 Pediatria,Cardiologia 320 15000000 HIT  
HospitalItaliano Cordoba 300 Traumatologia,Neurologia 190 9000000
```

pacientes.txt

Cada línea representa un paciente registrado:

```
codigo_hospital paciente_id dni fecha_ingreso diagnostico prioridad peso_kg  
La prioridad es un entero del 1 (crítico) al 5 (leve).
```

Ejemplo:

```
HGA 8001 30123456 20250310 Fractura 3 72.5 HIT 8002 27654321 20250401  
Apendicitis 1 65.0
```

derivaciones.txt

Representa conexiones entre hospitales (costo de traslado en minutos de ambulancia):

```
hospital_origen hospital_destino tiempo_minutos
```

Ejemplo:

```
HGA HIT 240 HGA HCO 180 HIT HCO 90
```

turnos.txt

Registra turnos médicos asignados:

```
codigo_hospital turno_id paciente_id medico_id fecha_turno especialidad  
duracion_min
```

Ejemplo:

```
HGA 5001 8001 201 20250315 Traumatologia 30 HIT 5002 8002 305 20250402  
Cirugia 90
```

Funcionalidades Requeridas

A. Gestión de Hospitales

- Mostrar la información completa de un hospital dado su código.
- Agregar un nuevo hospital al sistema.
- Eliminar un hospital existente (y reasignar sus pacientes activos al hospital más cercano disponible).
- Listar todos los hospitales ordenados por: capacidad de camas, cantidad de personal médico o presupuesto anual.
- Dado un hospital de origen y uno de destino, calcular la ruta de derivación de menor tiempo para trasladar un paciente crítico (implementar Dijkstra).
- Buscar hospitales que ofrezcan una especialidad específica y ordenarlos por disponibilidad de camas.

Requisito estructural: Utilizar una tabla de hashing con función hash propia para indexar hospitales por código. Justificar la elección de la función hash, indicar la estrategia de resolución de colisiones elegida (encadenamiento o direccionamiento abierto) y analizar el factor de carga.

B. Gestión de Pacientes y Turnos

1. Calcular el total de pacientes atendidos por hospital en un rango de fechas dado.
2. Detectar hospitales con sobrecarga: más de X ingresos en una semana o con ocupación superior al 90% de su capacidad.
3. Buscar todos los turnos de un paciente dado su DNI.
4. Implementar una cola de prioridad (min-heap) para gestionar la lista de espera de pacientes según su nivel de prioridad y fecha de ingreso. Deben poder realizarse operaciones de inserción, extracción del más prioritario y actualización de prioridad.
5. Dado un médico (por ID), listar todos sus turnos en orden cronológico.

Consigna de análisis: Para cada algoritmo implementado en esta sección, indicar su complejidad temporal en notación Big O, justificar la elección de la estructura de datos utilizada y argumentar si puede mejorarse con otra estructura o algoritmo alternativo.

C. Árbol de Decisión de Diagnósticos (Árbol Binario de Búsqueda)

El sistema debe mantener un árbol binario de búsqueda (BST) de diagnósticos ordenados por frecuencia de aparición en el sistema.

- Insertar un nuevo diagnóstico con su frecuencia inicial.
- Incrementar la frecuencia de un diagnóstico existente.
- Listar los diagnósticos en orden (inorder traversal).
- Encontrar el diagnóstico más frecuente (recorrido del árbol).
- Eliminar un diagnóstico del árbol manteniendo la propiedad BST.

- Calcular la altura del árbol y detectar si está desbalanceado (diferencia de alturas entre subárboles mayor a 2).

***Extensión:** Implementar rotaciones AVL para mantener el árbol balanceado automáticamente. Comparar la altura resultante entre el BST simple y el AVL con el mismo conjunto de datos.*

D. Optimización de Insumos con Backtracking

Cada ambulancia tiene una capacidad máxima de carga (en kg). Antes de una derivación, se debe decidir qué insumos médicos transportar para maximizar el valor clínico (urgencia/prioridad del insumo) sin superar la capacidad de la ambulancia.

- **Entrada:** lista de insumos con peso (kg) y valor clínico (entero).
- **Salida:** subconjunto de insumos que maximiza el valor total sin superar la capacidad.
- Implementar la solución con Backtracking puro.
- Implementar una versión con poda (branch & bound): descartar ramas cuando el valor restante posible no puede superar al mejor encontrado hasta el momento.
- Comparar experimentalmente el número de nodos explorados entre la versión sin poda y con poda para distintos tamaños de entrada ($n = 10, 20, 30$).
- Indicar la complejidad temporal de ambas versiones y discutir cuándo conviene usar programación dinámica en su lugar.

Condiciones de Entrega

- Exposición semanal de avances con el código funcionando.
- Entrega final con defensa oral del proyecto.
- Presentación del diseño UML: diagrama de clases completo con relaciones de herencia, composición y dependencia.
- Revisión del uso de estructuras de datos: se evaluará la justificación algorítmica de cada decisión de diseño.
- El código debe utilizar **camelCase** para variables y métodos. Los nombres de clases en **PascalCase**.
- Las clases deben implementarse de manera totalmente original y propia. **La detección de copias resultará en la pérdida de la regularidad de todos los involucrados.**
- Incluir un archivo **README.md** con instrucciones de compilación, descripción de cada módulo y ejemplos de uso.
- El sistema debe compilar sin warnings con **g++ -std=c++17 -Wall -Wextra**.